

Contents

guide 01 llama 3 herd_of_models	1
0. 先给新手的结论	1
1. 历史语境: 为什么这篇报告重要	2
2. 论文主张拆成问题、方法、证据	2
3. 总体架构图	3
4. 预训练数据: 数据质量仍然是第一杠杆	3
5. Data mix 是实验问题, 不是拍脑袋	4
6. 模型结构: 克制的 dense Transformer	4
7. Scaling Law: 为什么是 405B 和 15.6T	5
8. 训练基础设施: 16K H100 不是一句硬件描述	6
9. 4D Parallelism: TP、CP、PP、DP	6
10. 预训练 Recipe	6
11. Post-training: RM、RS、SFT、DPO 的六轮循环	7
12. Post-training 数据: 合成、过滤、难度和去重	8
13. 能力专项: Code、Math、Long Context、Tool Use	9
14. 实验证据链	9
15. Safety: 不是最后贴一层拒答	10
16. Inference: 405B 也要能服务	11
17. 和本仓库代码怎么连起来	11
18. 代码样例: Llama 3 缩小版 GQA	11
19. 代码样例: 训练 FLOPs 与 token 预算	12
20. AI agent 正确学习这篇报告的方法	12
21. 读完必须能闭卷回答	13
22. 一页复盘	13

guide_01_llama_3_herd_of_models

原论文: The Llama 3 Herd of Models

本地原文 PDF: [learning/pretraining-recipe/paper/01_llama_3_herd_of_models.pdf](#)

作者: Llama Team, AI @ Meta

年份: 2024

读法定位: 这是一份现代大模型“工厂级技术报告”, 不是单一算法论文。它把预训练、数据、模型结构、scaling law、集群训练、post-training、能力专项、安全、推理和多模态实验放在一份报告里。读懂它, 相当于读懂一个 2024 年顶级开放权重 LLM 是怎样被组织、训练、对齐、评估和部署的。

0. 先给新手的结论

Llama 3 的核心不是某个神奇结构, 而是三个杠杆一起发力: data、scale、managing complexity。Meta 训练了 8B、70B、405B 三个 dense Transformer 模型, 其中旗舰 405B 用约 $3.8e25$ FLOPs, 在 15.6T token 上预训练, 支持最高 128K 上下文。报告的关键观点是: 在这么大的工程规模上, 越复杂的模型/训练算法不一定越好; 稳定、可预测、可扩展的 dense Transformer, 加上更强的数据工程、scaling law、长上下文续训、迭代 post-training 和系统级安全, 能得到接近 GPT-4 级别的开放模型。

这篇报告要读成一条流水线:

```
raw data
-> cleaning / dedup / data mix
-> tokenizer
-> dense Transformer pre-training
-> long-context continued pre-training
-> final annealing + checkpoint averaging
-> reward model
-> rejection sampling
-> SFT
-> DPO
-> safety tuning + system guards
-> evaluation + inference optimization
```

如果只记一句话：Llama 3 证明了现代 LLM 能力不是“模型结构单点突破”，而是数据、规模、稳定训练、能力数据、评测闭环和安全系统共同堆出来的工程结果。

1. 历史语境：为什么这篇报告重要

Llama 1 和 Llama 2 已经证明：开放权重模型可以在研究社区和产业里形成巨大影响。但到 2024 年，竞争已经不再只是“有没有一个 7B/70B 模型”。强模型需要同时具备：

- 多语言能力。
- 代码能力。
- 数学和推理能力。
- 工具使用能力。
- 长上下文能力。
- 安全可控能力。
- 接近闭源前沿模型的 benchmark 和人评表现。

这就把问题从“训练一个 Transformer”升级成“管理一个模型族”。论文标题里的 herd 很关键：Llama 3 不是单个模型，而是一组模型，包括 pre-trained / instruct 版本、8B / 70B / 405B 尺度、长上下文、多语言、工具使用，以及 Llama Guard 3 这样的安全分类器。

当时有两种路线可以选：

- 用更复杂结构，例如 MoE，理论上训练/推理效率更好，但系统和稳定性更难。
- 用标准 dense Transformer，架构上克制，把复杂度主要放到数据、并行、post-training、评测和安全管线。

Llama 3 选择了第二条路。报告反复强调 managing complexity：规模已经足够大时，稳定、可调试、可预测，比追求复杂结构更重要。

2. 论文主张拆成问题、方法、证据

问题：现代 foundation model 需要在通用知识、代码、数学、多语言、长上下文、工具使用和安全之间同时达到高水平。单纯增加参数或 token 不够，训练和对齐流程必须可扩展、可预测、可评估。

方法：使用相对标准的 dense Transformer，主要通过更大的多语言高质量数据、 $3.8e25$ FLOPs 级训练、scaling law 选择 405B/15.6T 规模、4D parallelism 保证训练可运行、long-context continued pre-training 推到 128K、六轮 post-training 做 SFT + DPO，并用 Llama Guard 3 / Prompt Guard / Code Shield 做系统级安全。

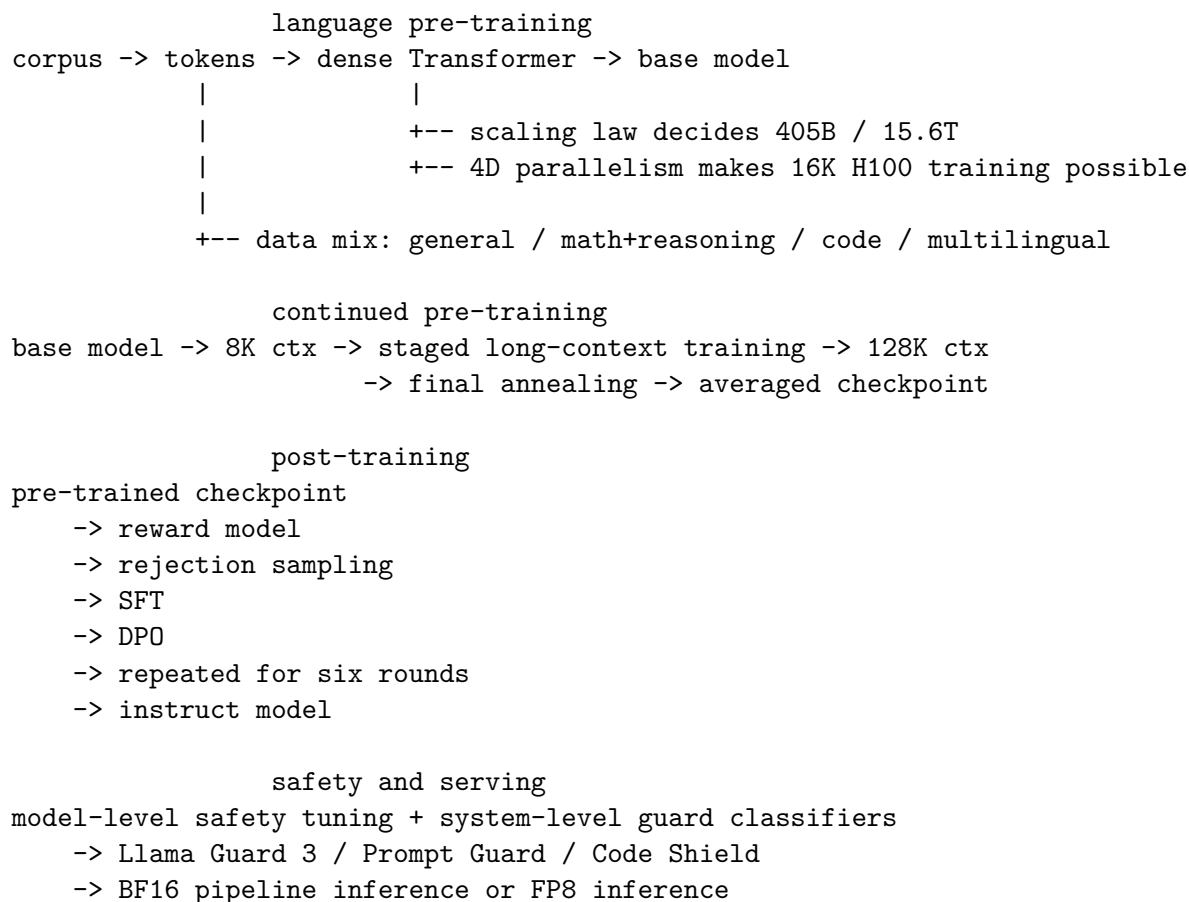
证据：报告提供了大量 benchmark、人类偏好评测、安全评测和效率评测。Llama 3 405B 在许多任务上接近 GPT-4 / GPT-4o / Claude 3.5 Sonnet；8B 和 70B 在同尺寸开放模型里非常强；长上下文在

needle/infinitebench 等任务上表现强；工具使用、代码、数学、多语言都有专项数据和专项评测。

局限：评测并不完全可复现，部分 benchmark 和安全数据是内部的；训练成本极高；多语言安全只覆盖核心语言；长上下文、工具调用、红队攻击仍有残余风险；405B 推理需要复杂并行和量化；模型能力来自海量工程投入，不是个人可以完整复刻的 recipe。

3. 总体架构图

Llama 3 把 foundation model development 分成两个主阶段：pre-training 和 post-training。多模态是额外实验，本导读重点放在语言模型主线。



这张图要和上一篇 The Pile 连起来：The Pile 解决的是“开放预训练语料怎样构建”；Llama 3 解决的是“当数据、模型、后训练、安全和推理都达到工业规模时，整条链路怎样协同”。

4. 预训练数据：数据质量仍然是第一杠杆

报告第一节就说，Llama 3 的三个关键杠杆是 data、scale、managing complexity。数据排第一。

预训练语料覆盖截至 2023 年底的多种来源。Meta 对每个数据源做清洗、去重和过滤，移除含大量 PII 的域名和已知成人内容域。Web 数据处理包括：

- PII 和 safety 过滤。
- 自研 HTML parser，目标是在 boilerplate removal 和 content recall 之间取得更好平衡。
- 对数学和代码网页做特殊处理，保留结构；保留 image alt attribute，因为数学内容可能以图片和 alt 文本同时存在。
- 实验发现对主要由网页训练的模型来说，markdown markers 有害，因此移除 markdown 标记。

- URL-level、document-level、line-level 多轮去重。
- 用 token-distribution KL divergence 过滤 outlier token 过多的文档。
- 使用 fastText 等分类器挑选高质量 token。

最终数据混合大致是：

- general knowledge: 50%。
- math and reasoning: 25%。
- code: 17%。
- multilingual: 8%。

这和很多简化讲法里的 “web/code/other” 不同。论文的真实重点是：数据域是围绕能力目标组织的，不只是围绕来源组织的。

5. Data mix 是实验问题，不是拍脑袋

Llama 3 用两类工具决定 data mix。

第一类是 knowledge classification。作者训练分类器识别网页中的知识域，并下采样过度代表的类别。这个动作的意义是：不是所有 web token 都等价。某些类别过多，会占据训练预算，但不一定提升目标能力。

第二类是 scaling law experiments。作者训练多个小模型，比较不同 data mix 对 downstream benchmark 的预测，再选择候选 mix。接着再训练更大的模型验证。这个过程迭代多次。

直觉可以写成：

```
candidate mix
-> train small scaling-law models
-> predict large-model benchmark behavior
-> choose better mix
-> train larger validation model
-> update mix
```

这点非常重要：数据配比不是 “读完论文记住 50/25/17/8” 就完了。真正的 recipe 是用小规模实验预测大规模行为。

论文还用 annealing data 来评估小型领域数据集价值。具体做法是：拿一个训练到 50% 的 Llama 3 8B，把学习率在 40B token 上线性降到 0，其中 30% 权重给新数据集，70% 保留默认 mix。这个方法比为每个小数据集做完整 scaling law 实验便宜得多。

6. 模型结构: 克制的 dense Transformer

Llama 3 架构和 Llama / Llama 2 相比没有大幅偏离。论文甚至明确说，性能提升主要来自数据质量、多样性和训练规模，而不是结构奇技。

关键结构参数：

- 8B: 32 layers, dim 4096, FFN 14336, 32 heads, 8 KV heads, peak LR 3e-4。
- 70B: 80 layers, dim 8192, FFN 28672, 64 heads, 8 KV heads, peak LR 1.5e-4。
- 405B: 126 layers, dim 16384, FFN 53248, 128 heads, 8 KV heads, peak LR 8e-5。

共同设置：

- activation: SwiGLU。
- vocabulary size: 128K。

- positional embeddings: RoPE, base frequency $\theta = 500000$.
- attention: grouped query attention, GQA.

GQA 的张量级直觉:

hidden x : $[B, T, d_{\text{model}}]$

$Q = x W_q \rightarrow [B, n_q \text{ heads}, T, \text{head_dim}]$

$K = x W_k \rightarrow [B, n_{kv} \text{ heads}, T, \text{head_dim}]$

$V = x W_v \rightarrow [B, n_{kv} \text{ heads}, T, \text{head_dim}]$

$n_q \text{ heads} > n_{kv} \text{ heads}$

K/V are shared by groups of query heads

KV cache size shrinks roughly by $n_{kv} \text{ heads} / n_q \text{ heads}$

405B 有 128 个 attention heads, 但只有 8 个 key/value heads。这样解码时 KV cache 大幅下降, 对推理特别重要。论文也指出 GQA 让 context parallelism 的 all-gather K/V 成本更小, 因为 K/V 张量远小于 Q 张量。

Tokenizer 也很关键。Llama 3 用 128K token vocab, 其中 100K 来自 tiktoken, 另加 28K 支持非英语语言。相比 Llama 2 tokenizer, 英文样本压缩率从 3.17 characters/token 提升到 3.94 characters/token。这等于在同样 token 预算下 “读” 更多文本。

7. Scaling Law: 为什么是 405B 和 15.6T

报告用 scaling law 做两件事:

1. 在给定训练计算预算下, 选择旗舰模型的参数量和 token 数。
2. 预测 downstream benchmark 表现, 而不只是预测 validation loss。

实验范围: 作者用 $6e18$ 到 $1e22$ FLOPs 的预算, 训练 40M 到 16B 参数的小模型。每个 compute budget 下, 用不同 token 数训练, 得到 IsoFLOPs 曲线。对每条曲线, loss 关于 training tokens 呈类似 U 形; 曲线最低点就是这个 compute budget 下的 compute-optimal 配置。

然后他们假设 compute budget C 和最优训练 token 数 $N_{\text{opt}}(C)$ 满足:

$N_{\text{opt}}(C) = A * C^{\alpha}$

拟合得到:

$\alpha = 0.53$

$A = 0.29$

把这个关系外推到 $3.8e25$ FLOPs, 建议训练约 402B 参数模型、16.55T token。最终选择 405B、15.6T token, 非常接近这个预测。

为什么这很重要? 因为 405B 训练成本巨大, 不能靠 “先训一个看看” 试错。Scaling law 是训练前的导航仪。论文还进一步用两阶段方法预测 benchmark:

training FLOPs

-> normalized NLL of correct benchmark answer

-> task accuracy through sigmoid relation

作者说这种跨四个数量级外推在 ARC Challenge 上相当准确, 只是略低估最终 405B 表现。

8. 训练基础设施: 16K H100 不是一句硬件描述

405B 预训练最多使用 16K H100 GPU, 每张 80GB HBM3、700W TDP。训练集群使用 RoCE / Infiniband、400Gbps interconnect、Tectonic 分布式存储。存储规模达到 240PB, 持续吞吐 2TB/s, 峰值 7TB/s。

这部分最值得学的是: 大模型训练不是只要 GPU 多。它还需要:

- checkpoint 写入不把训练暂停太久。
- 网络拓扑和并行策略匹配。
- collective communication library 能处理高延迟网络。
- 自动故障诊断和恢复。
- 对 silent data corruption、straggler、温度导致吞吐波动有监控。

论文报告在一个 54 天窗口中有 466 次 job interruptions, 其中 419 次是 unexpected。约 78% unexpected interruption 来自确认或疑似硬件问题。即便如此, 他们仍然达到超过 90% effective training time。

这句话对学习者很扎心但很重要: 到 16K GPU 规模, 训练稳定性本身就是算法的一部分。

9. 4D Parallelism: TP、CP、PP、DP

Llama 3 405B 需要四维并行:

- TP: tensor parallelism, 把单个权重张量切到多个设备。
- CP: context parallelism, 把长序列在 sequence 维度切分。
- PP: pipeline parallelism, 把层按 stage 切分。
- DP/FSDP: data parallelism, 同时 shard optimizer states、gradients, 并并行处理数据。

并行维度顺序是:

[TP, CP, PP, DP]

原因是内层并行需要最高带宽和最低延迟, 尽量放在同 server / NVLink 里; 外层 DP 能容忍更高延迟, 可以跨更远网络。

关键配置:

- 8192 GPUs: TP 8, CP 1, PP 16, DP 64, seq len 8192, 16M tokens/batch, MFU 43%。
- 16384 GPUs: TP 8, CP 1, PP 16, DP 128, seq len 8192, 16M tokens/batch, MFU 41%。
- 16384 GPUs long context: TP 8, CP 16, PP 16, DP 8, seq len 131072, 16M tokens/batch, MFU 38%。

这里的 MFU 是 BF16 Model FLOPs Utilization。38-43% 在这种规模上很强。

Context parallelism 的细节也值得看。长上下文时, 序列长度到 128K, 自注意力内存压力非常大。Llama 3 把 input sequence 切成 $(2 \times \text{CP})$ chunks, 让每个 CP rank 拿两个 chunk 做负载均衡。它没有采用 ring-like overlap, 而是先 all-gather K/V, 再对本地 Q chunk 算 attention。因为 GQA 让 K/V 比 Q 小得多, 所以 all-gather 的 $(O(S))$ 开销相对 attention 的 $(O(S^2))$ 可接受。

10. 预训练 Recipe

405B 预训练分三个阶段:

```
initial pre-training
-> long-context pre-training
-> final annealing
```

Initial pre-training 设置:

- optimizer: AdamW。
- peak learning rate: $8e-5$ 。
- warmup: 8000 steps。
- cosine decay: 到 $8e-7$ 。
- total: 1,200,000 steps。
- 初始 batch: 4M tokens, sequence length 4096。
- 252M tokens 后升到 8M batch、8192 sequence。
- 2.87T tokens 后升到 16M batch。

这个 batch schedule 的理由是: 早期小 batch 更稳, 后期大 batch 更高效。论文说这个 recipe 很稳定, loss spikes 很少, 不需要干预训练发散。

训练中还会调整 data mix:

- 增加非英语数据, 提高 multilingual 表现。
- 上采样数学数据, 提高数学推理。
- 加入更新的 web data, 推进 knowledge cutoff。
- 下采样后来识别出的低质量子集。

Long-context pre-training:

- 不在训练早期直接用长序列, 因为 self-attention 计算随 sequence length 二次增长。
- 从 8K 逐步增加到 128K, 共六个阶段。
- 判断是否适应新长度, 主要看短上下文评测是否恢复, 以及 needle-in-a-haystack 是否在该长度完美。
- 这一阶段用约 800B tokens。

Final annealing:

- 最后 40M tokens, learning rate 线性退火到 0。
- 保持 128K context。
- 上采样非常高质量数据。
- 对 annealing 期间 checkpoint 做 Polyak averaging, 得到最终 pre-trained model。

11. Post-training: RM、RS、SFT、DPO 的六轮循环

Llama 3 的 post-training 不是一次 SFT。它是六轮迭代。

current model

- > collect preference data
- > train reward model
- > rejection sampling
- > SFT on selected/generated targets
- > DPO on latest preference batches
- > model averaging
- > next round

Reward model 用 pre-trained checkpoint 初始化, 训练在 preference data 上。与 Llama 2 相比, 它移除了 margin term, 因为数据扩大后 margin 带来的收益递减。样本可能有三档排序:

edited > chosen > rejected

SFT 使用标准 cross entropy, 只在 target tokens 上算 loss, prompt tokens 被 mask。最大模型 SFT 学习率约 $1e-5$, 训练 8.5K 到 9K steps。

DPO 用最新一批 preference data, 使数据分布更贴近当前 policy。作者也探索过 PPO, 但发现对大模型而言 DPO 计算更省、效果更好, 尤其在 IFEval 等指令遵循 benchmark 上。Llama 3 的 DPO 设置:

- learning rate: 1e-5。
- beta: 0.1。
- mask formatting tokens, 包括 header / termination tokens。
- 加 chosen sequence 的 NLL 正则, 系数 0.2。

DPO 的基本目标可以理解为: 让 chosen response 相对 rejected response 的 log-prob 差距变大, 同时参考模型提供约束, 避免 policy 乱漂。

简化公式:

$$L_{DPO} = -\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right] \right)$$

Llama 3 的额外 NLL 正则可以写成:

$$L = L_{DPO} + 0.2L_{NLL}(y_w)$$

这个正则不是数学装饰。论文说它能稳定 DPO, 保持格式, 防止 chosen response 的 log probability 下降。

12. Post-training 数据: 合成、过滤、难度和去重

Preference data 由多轮标注收集。标注者比较两个模型回答, 给出偏好强度: significantly better、better、slightly better、marginally better。部分样本还会编辑 preferred response, 形成 edited > chosen > rejected。

SFT 数据包括:

- 人类标注 prompt 上经过 rejection sampling 选择的回答。
- 针对能力的合成数据。
- 少量人工 curated data。

Final SFT mix 的大类比例:

- General English: 52.66% examples。
- Code: 14.89%。
- Multilingual: 3.01%。
- Exam-like: 8.14%。
- Reasoning and tools: 21.19%。
- Long context: 0.11%。

注意 long context 只有 0.11% examples, 但平均 token 很长, 约 38K tokens。样本数占比不等于 token 占比, 更不等于能力影响。

数据质控非常细:

- rule-based 清洗, 去掉过多 emoji、感叹号、过度道歉等模式。
- topic classification, 把样本分到粗粒度和细粒度主题。
- quality scoring, 结合 reward model 和 Llama-based scoring。
- difficulty scoring, 用 Instag 和 Llama-based 难度评分。

- semantic deduplication, 按 RoBERTa embedding 聚类, 在簇内按 quality score x difficulty score 排序, 再贪心保留不太相似的样本。

这说明现代 post-training 数据不是“越多越好”, 而是质量、难度、主题覆盖和多样性之间的多目标优化。

13. 能力专项: Code、Math、Long Context、Tool Use

Code

Llama 3 为 code 训练了 code expert: 从主预训练分支继续训练 1T token, 其中超过 85% 是 code。之后做 long-context fine-tuning 到 16K, 再用 code-focused SFT/DPO 对齐。合成数据超过 2.7M examples, 核心质量信号来自执行反馈、parser、linter、unit tests、container execution。约 20% 初始错误解法能通过自我修正变正确。

Multilingual

为了更好收集非英语标注, 作者训练 multilingual expert: 从预训练分支继续训练, 数据 mix 里 90% 是 multilingual tokens。支持的核心语言包括 English、German、French、Italian、Portuguese、Hindi、Spanish、Thai。多语言 SFT 数据来源包括人工标注、其他 NLP 任务改写、rejection sampling、少量翻译 reasoning data。作者特别警惕 translationese、文化偏差和语言不匹配。

Math and Reasoning

难点包括: 复杂题 prompt 稀缺、ground-truth chain-of-thought 稀缺、模型生成中间步骤可能错、训练和推理使用方式不一致。方法包括数学技能 taxonomy、从预训练数学文本转 QA、生成 step-by-step 解法、用正确答案过滤、self-verification、outcome/stepwise reward model、MCTS、Python code execution 反馈, 以及从错误中纠正。

Long Context

预训练把上下文扩到 128K, 但 post-training 如果只用短上下文数据, 会让长上下文能力退化。长上下文 SFT 主要靠合成数据: 长文 QA、层级摘要、repo-level code reasoning。作者做 ablation 后发现, 把 0.1% long-context synthetic data 混入原短上下文数据, 可以在短上下文和长上下文 benchmark 之间取得较好平衡。

Tool Use

Llama 3 学会 Search、Python interpreter、Wolfram Alpha, 以及 zero-shot function calling。工具使用数据包括单步、多步、文件上传、多轮 function calling。核心格式是:

```
system prompt
user prompt
assistant tool call
tool output
assistant final answer
```

这和 agent 课程直接相连: 工具调用不是靠“模型自然会用”, 而是靠协议、数据、工具输出、标注偏好和安全约束共同塑形。

14. 实验证据链

报告结果非常多, 不要背表格, 要抓证据类型。

总览表

Llama 3 405B 在 MMLU、MMLU-Pro、HumanEval、GSM8K、MATH、ARC Challenge、GPQA、BFCL、long-context、多语言等任务上与 GPT-4 / GPT-4o / Claude 3.5 Sonnet 竞争。8B 和 70B 在同尺寸类别里通常领先。

预训练模型

作者评估 reading comprehension、code、commonsense、math/reasoning、general 等任务。405B 在 MMLU 85.2、MMLU-Pro 61.6、GSM8K 89.0、MATH 53.8、ARC-C 96.1、BBH 85.9 等预训练评测上很强。8B/70B 也显著优于同尺寸开放模型。

Post-trained 模型

关键示例：

- HumanEval pass@1: 8B 72.6, 70B 80.5, 405B 89.0。
- MGSM: 8B 68.9, 70B 86.9, 405B 91.6。
- Long context multi-needle: 8B 98.8, 70B 97.5, 405B 98.1。
- BFCL: 8B 76.1, 70B 84.8, 405B 88.5。

Human eval

405B 与 GPT-4 大体相当；相对 GPT-4o 和 Claude 3.5 Sonnet 有赢有输。报告特别指出，人评受 tone、verbosity、response structure 等细微因素影响，这也是 post-training 需要反复迭代的原因。

Contamination

预训练评测使用 8-gram overlap 检测污染。作者承认 contamination analysis 仍是开放问题，会有 false positive / false negative。这一点对学习很重要：强 benchmark 结果必须搭配污染分析一起看。

15. Safety: 不是最后贴一层拒答

Llama 3 的安全从预训练开始，包括 PII/unsafe domain 过滤和 memorization 测试。报告用 rolling hash index 抽取不同频率的 n-gram，测模型是否 verbatim 生成 ground truth continuation。405B 的平均 verbatim memorization 率在 selected scenarios 下约为：

- English 50-gram: 1.13%。
- All 50-gram: 1.03%。
- All 1000-gram: 3.91%。

Safety finetuning 同时优化两个指标：

- VR: violation rate, 模型违反安全政策的比例。
- FRR: false refusal rate, 模型错误拒绝安全请求的比例。

安全数据包括 adversarial prompts 和 borderline prompts。边界样本很关键，因为只训练拒绝会让模型过度拒答。报告还说明，小模型需要更高比例 safety data 才能达到类似安全表现，大模型更容易区分 adversarial 和 borderline。

系统级安全包括：

- Llama Guard 3: 基于 Llama 3 8B 的安全分类器，检测输入/输出是否违反政策。
- Prompt Guard: 检测 direct jailbreak 和 indirect prompt injection。
- Code Shield: 用静态分析检测不安全代码。

Llama Guard 3 平均减少约 65% violation，但会增加 false refusal。这就是安全工程的典型 trade-off：更严的 guard 会更安全，也更容易误伤正常请求。

16. Inference: 405B 也要能服务

405B BF16 参数放不进单机 8 张 H100。推理时采用跨 16 GPU、两台机器的 parallelism：机内用高带宽 NVLink 做 tensor parallelism，跨机器用 pipeline parallelism。推理没有 backward pass，因此 pipeline bubble 不像训练时那么严重，可以用 micro-batching 提升吞吐。

FP8 quantization 是另一个关键。作者对 feedforward network 里的大多数矩阵乘法做 FP8，因为 FFN 约占推理计算 50%。注意他们没有量化 self-attention 参数，并且做了几个修正：

- 不量化第一层和最后一层 Transformer。
- 对 dynamic scaling factor 设置上限 1200，避免高 perplexity token 造成 underflow 和解码错误。
- 使用 row-wise quantization，比 tensor-wise 更细粒度。

一个很好的评测点是：标准 benchmark 可能看不出 FP8 的坏响应。作者用 100K 个 BF16/FP8 回答的 reward-model score distribution 对比，来判断输出分布是否漂移。这是工程评估里非常值得学的一点。

17. 和本仓库代码怎么连起来

本仓库不能复刻 Llama 3，但能缩小成可跑的教学版。

关键文件：

- learning/pretraining-recipe/src/data_mixture.py 演示 data mix、curriculum、final annealing mix。注意其中 llama3 配比是教学近似；论文真实 summary 是 50% general、25% math/reasoning、17% code、8% multilingual。
- learning/pretraining-recipe/src/phi_tiny_model.py 实现 Pre-RMSNorm、GQA、RoPE、SwiGLU、tied embedding。它是 Llama/Phi 风格 dense Transformer 的缩小版。
- learning/pretraining-recipe/src/init_schedule.py 包含 warmup + cosine、WSD、inverse sqrt、muP lr scaling。Llama 3 主要使用 warmup + cosine，最后有 annealing。
- learning/pretraining-recipe/src/dataset_shards.py 用 numpy memmap 模拟 token shard 与 batch sampling。对应大规模训练中的 shard / resume / streaming 数据路径。
- learning/pretraining-recipe/src/training_loop.py 包含 AdamW、weight decay 分组、bf16 autocast、grad clip、loss spike tracker、checkpoint。
- learning/pretraining-recipe/src/distillation.py 对应 Llama 3 用旗舰模型提升小模型质量这一类思想，也对应后续合成数据/teacher-student 路线。

18. 代码样例: Llama 3 缩小版 GQA

本仓库的 GQA 逻辑可以对应论文的 KV heads 设计：

B, T, H = x.shape

```
q = q_proj(x).view(B, T, n_head, head_dim).transpose(1, 2)
k = k_proj(x).view(B, T, n_kv_head, head_dim).transpose(1, 2)
v = v_proj(x).view(B, T, n_kv_head, head_dim).transpose(1, 2)
```

```
repeat = n_head // n_kv_head
k = k.repeat_interleave(repeat, dim=1)
v = v.repeat_interleave(repeat, dim=1)
```

```
out = scaled_dot_product_attention(q, k, v, is_causal=True)
```

在 toy 模型里, $n_{\text{head}}=16$ 、 $n_{\text{kv_head}}=4$ 。在 Llama 3 405B 里, $n_{\text{head}}=128$ 、 $n_{\text{kv_head}}=8$ 。机制一样: 多个 query heads 共享更少的 K/V heads, 从而减小 KV cache。

19. 代码样例: 训练 FLOPs 与 token 预算

Llama 3 报告中的训练 FLOPs 可以用粗略公式理解:

$$C \approx 6ND$$

其中 (N) 是参数量, (D) 是训练 token 数。比如 405B、15.6T:

```
def estimate_flops(n_params, n_tokens):  
    return 6 * n_params * n_tokens
```

```
flops = estimate_flops(405e9, 15.6e12)  
print(flops)  # about 3.79e25
```

这和论文的 $3.8e25$ FLOPs 对齐。这个小函数在 `common.py` 里已经有。

对比 Chinchilla 1:20:

```
def chinchilla_optimal_tokens(n_params):  
    return 20 * n_params
```

```
print(chinchilla_optimal_tokens(8e9))  # 160B tokens
```

但 Llama 3 小模型被训练得远超 compute-optimal tokens, 因为小模型面向固定 inference budget, overtraining 可以让同样参数量的模型更强。这是论文第一节特别点出的策略。

20. AI agent 正确学习这篇报告的方法

这篇报告很长, 最容易出现 “agent 给你一堆摘要, 你觉得自己懂了” 的错觉。正确用法是让 agent 帮你做结构化追问, 而不是代替你读。

建议四轮:

第一轮: 架构地图

请只根据我给你的 Llama 3 笔记提问。

一次一个问题, 按 data -> architecture -> scaling law -> training infra -> recipe -> post-training -> safety 的顺序。我回答后, 你指出我漏掉的数字或证据。

第二轮: claim/evidence 对照

请为 Llama 3 建 claim/evidence 表。

每行包含 claim、论文证据、可能混淆因素、本仓库对应文件。
不要写泛泛总结, 必须指向一个表、公式、设置或实验。

第三轮: 本地 toy 实验

让 agent 陪你跑这些最小实验:

```
pytest learning/pretraining-recipe/src/tests/test_pretraining.py
python learning/pretraining-recipe/src/phi_tiny_model.py
python learning/pretraining-recipe/src/data_mixture.py
python learning/pretraining-recipe/src/init_schedule.py
```

每跑一个实验, 你自己写五句话:

1. 它对应 Llama 3 哪个机制。
2. 输入/输出张量是什么。
3. 改哪个参数会影响什么。
4. toy 实验和真实论文差在哪里。
5. 我现在能闭卷画出哪张图。

第四轮: 闭卷复述

要求 agent 不给答案, 只打分:

我用 300 字闭卷复述 Llama 3。

请按 data、scale、complexity、post-training、safety 五项各给 0-2 分。

只指出缺口, 不要重写我的答案。

21. 读完必须能闭卷回答

1. Llama 3 说的三个关键杠杆是什么?
2. 为什么 Llama 3 选择 dense Transformer, 而不是 MoE?
3. 预训练数据最终 mix 的 50/25/17/8 分别是什么?
4. 为什么 data mix 要靠 scaling law 和小模型实验决定?
5. Llama 3 405B 的层数、hidden dim、heads、KV heads、vocab、RoPE theta 是多少?
6. $N_{opt}(C) = A * C^{\alpha}$ 在论文里怎么用? α 和 A 是多少?
7. 为什么小模型可以训练远超 compute-optimal token?
8. 4D parallelism 的四个维度分别解决什么问题?
9. 预训练三阶段是什么? long-context 阶段用了多少 token?
10. DPO 在 Llama 3 里做了哪些稳定性修改?
11. code/math/tool/long-context 的合成数据各自靠什么质量信号过滤?
12. Llama Guard 3 降低了什么风险, 又带来什么代价?
13. FP8 量化为什么不能只看 benchmark 分数?
14. 如果在本仓库复现一个 toy 机制, 你会打开哪个文件、改哪个参数、看哪个指标?

22. 一页复盘

Llama 3 是现代 LLM 工程的系统报告。它的核心不是架构大改, 而是 data、scale、managing complexity: 用更强的数据清洗和混合, 在 15.6T token 上训练 405B dense Transformer; 用 scaling law 从小模型外推到 $3.8e25$ FLOPs 的旗舰训练; 用 GQA、128K vocab、RoPE theta 500000 保持架构稳定; 用 4D parallelism 和 16K H100 让训练可运行; 先做 8K 预训练, 再用约 800B token 续训到 128K, 最后 40M token annealing 和 checkpoint averaging; post-training 用 RM、rejection sampling、SFT、DPO 六轮迭代, 并用合成数据、执行反馈、reward model、语义去重强化代码、数学、工具和长上下文; 安全上结合预训练过滤、safety SFT/DPO、red teaming、Llama Guard 3、Prompt Guard、Code Shield; 推理上用 pipeline parallelism 和 FP8 量化服务 405B。它告诉你: 前沿 LLM 是工程系统, 不是单个 trick。